Threshold

A plain-language guide to the project, the payment flow, and the demo

Read this once before presenting. It is written for understanding, not for impressing people with jargon.

Threshold, Explained Like a Friend

What this project is

Threshold is a small marketplace for AI capabilities that software agents can use when they get stuck.

Imagine an automated build agent running tests. It finds a bug, but it does not know how to fix it. Instead of having a human search for a service, Threshold lets the agent:

1. Describe the problem.
2. Look through a catalog of available capabilities.
3. Choose the service that matches the problem.
4. Pay for one request with a tiny USDC payment.
5. Send the bug and code to the paid provider.
6. Receive a structured diagnosis and code patch.

The important idea is that the agent does not only ask an AI model for an answer. It discovers and pays for a separate capability through a real machine-to-machine payment flow.

The one-sentence pitch

Threshold lets a software agent discover and pay for the exact debugging capability it needs, using x402 payments on Algorand, and receive a machine-readable fix instead of waiting for a human.

The problem

Today, an AI coding agent can find a bug but often cannot finish the job. It may need a security specialist, code reviewer, debugger, or another expert service. Usually a human has to find that service, understand its API, arrange payment, and pass the data to it.

That does not scale well when agents are working by themselves. Agents need a standard way to discover capabilities and pay for individual API calls.

Threshold is a prototype of that missing layer.

A simple story

Agent A is a build runner. It detects this real-looking production bug:

- A profile cache stores data using only userId.
- The same user ID can exist in multiple tenants.

- A request from acme-us fills the cache.
- A later request from acme-eu gets the US profile by mistake.
- That is a tenant-isolation and data-leak risk.

Agent A cannot safely fix the issue itself. It asks Threshold for help. Threshold discovers the incident-resolution capability, pays the provider, and gets this kind of patch back:

```
--- a/src/profile-cache.ts
+++ b/src/profile-cache.ts
@@
-  const key = userId;
+  const key = `${tenantId}:${userId}`;
```

The change is small, but the problem is realistic: cache keys must include every piece of identity that affects the data.

What “agent” means here

An agent is software that can make decisions and take actions toward a goal. In this project, Agent A is represented by the backend flow and browser trace. It is not a human clicking through every API choice.

Agent B is a provider. It owns a capability, such as incident resolution, and returns a result when paid.

The names are a useful way to understand the roles:

- Agent A: the caller that has a blocked task.
- Threshold: the discovery and payment gateway.
- Agent B: the provider that owns the useful capability.

What an API is

An API is a controlled doorway into software. One program sends an HTTP request to a URL, and another program returns data.

For example:

```
POST /api/resolve-incident
```

means: send a POST request to the incident-resolution service. The request contains the runtime, language, error, source files, and constraints. The response contains a diagnosis, confidence score, patch, and verification command.

What the catalog does

The catalog is a directory of capabilities. Threshold exposes it at:

```
GET /api/catalog
```

The current catalog includes:

- Incident Resolution: debugging, patch generation, incident analysis.
- AI Code Review: correctness, security, and maintainability review.
- AI Text Summarizer: turning long text into a concise summary.

Each entry contains a public name, description, endpoint, price, network, capabilities, and provider wallet address.

The incident flow looks at the catalog and chooses the entry containing bug-resolution. It also reports the alternatives it rejected. This matters because discovery should be visible: the system should demonstrate that it inspected available capabilities instead of silently calling one hardcoded URL.

What x402 means

HTTP normally has status code 200 for success and 404 for not found. x402 adds a payment meaning to HTTP status code 402: Payment Required.

The useful mental model is:

1. The caller asks for a protected resource without payment.
2. The server returns 402 plus payment instructions.
3. The caller builds and signs a payment.
4. A facilitator verifies and settles it.
5. The caller retries the original request with proof of payment.
6. The server accepts the request and returns the service result.

This is useful for agents because payment is part of the protocol instead of a separate human checkout screen.

What Algorand is doing

Algorand is the blockchain used for the payment. The project uses USDC as the asset being transferred.

For the judging setup, the project uses Algorand TestNet:

- Network: Algorand TestNet.
- Asset: USDC ASA 10458941.
- Price: $0.001.
- On-chain amount: 1000 micro-USDC.

TestNet is a practice network. It uses real blockchain mechanics but test funds, so the demo can be repeated without spending real money. The code also contains MainNet configuration, but the hackathon requirement is explicitly TestNet, so the judging setup should keep X402_NETWORK=testnet.

An ASA is an Algorand Standard Asset. It is Algorand's identifier for tokens such as USDC. The number 10458941 identifies the TestNet USDC asset used here.

What the facilitator does

The facilitator is GoPlausible:

```
https://facilitator.goplausible.xyz
```

The facilitator is a trusted service used by the x402 server flow. It checks that the payment payload is valid and settles the Algorand transaction. The application does not pretend that payment happened just because a button was clicked. It waits for the facilitator response and uses the returned settlement information.

A successful settlement response includes data such as:

- success: true
- payer wallet
- transaction ID
- Algorand network

The UI exposes the facilitator hostname in the settlement inspector so a judge can see which service handled the payment.

What AVM means

AVM means Algorand Virtual Machine. In this project, @x402/avm provides the Algorand-specific x402 payment scheme and signer integration.

The important packages are:

- @x402/avm: Algorand payment scheme and signer support.
- @x402-avm/extensions: discovery metadata extensions.
- @x402/core: shared x402 protocol types and server logic.
- @x402/fetch: client wrapper that reacts to 402 responses.
- @x402/hono: Hono middleware that protects routes with payment requirements.

The complete live flow

1. The browser starts the flow

The button calls:

```
POST /api/agent/run-payment-flow
```

The response is Server-Sent Events, commonly called SSE. SSE is a long-lived HTTP response where the server sends one event whenever something happens.

2. Agent A reports the incident

The first event contains the real incident fixture: the tenant cache bug, affected files, error message, and constraints.

3. Agent A queries the catalog

The backend calls:

```
GET /api/catalog
```

It selects resolve-incident because the incident needs debugging and patch generation. The event includes the alternatives that were rejected.

4. Threshold makes an unpaid request

The backend sends the incident to the protected provider endpoint without payment:

```
POST /api/resolve-incident
```

The server returns HTTP 402. The response includes the required network, asset, amount, and provider wallet.

5. The server loads the payer

The mnemonic stays on the server in .env. It is never sent to the browser. The server derives a signer and checks that the derived wallet matches AVM_PAYER_ADDRESS.

This check prevents a very confusing failure where the configured mnemonic and displayed payer address refer to different wallets.

6. The x402 client prepares payment

The server creates an x402Client, registers ExactAvmScheme, and connects the signer to the Algorand TestNet network identifier.

“Exact” means the request has an exact price and exact receiver. The payer is not making an open-ended offer.

7. The paid retry happens

wrapFetchWithPayment handles the 402 response. It constructs the payment payload, signs the Algorand transaction, sends the payment through the facilitator, and retries the original request with payment proof.

8. GoPlausible settles the payment

The facilitator verifies and settles the payment. The backend extracts the settlement response and emits it into the live event stream.

9. The provider returns a structured fix

The provider endpoint sends the incident to Gemini with a strict JSON response shape. The result contains:

- Diagnosis: the root cause.
- Confidence: how certain the provider is.
- Patch: file path plus unified diff.
- Verification: test command and expected result.

The browser renders the actual diff returned by the provider.

10. The browser shows evidence

The execution trace shows real timestamps and event details. The settlement inspector shows the real payer, provider, network, asset, amount, facilitator, and transaction ID.

The transaction link uses:

```
https://lora.algokit.io/testnet/transaction/<transaction-id>
```

A judge can click it and inspect the actual Algorand transaction.

What SSE means in plain English

SSE is like keeping a phone call open between the browser and server. The browser does not ask every few seconds whether something happened. The server pushes an event as soon as the next real step completes.

That is why the trace can show the flow progressively:

- incident detected
- capability selected
- payment required
- wallet loaded
- payment signing requested
- facilitator settlement received
- paid retry accepted
- structured patch returned

There is no timer that pretends payment took place. If the facilitator is slow, the screen stays in a pending state.

The failure demo

The “Underfunded wallet” toggle generates an empty wallet and sends it through the same payment path.

The flow still performs discovery, receives a real 402, loads the empty wallet, and attempts the paid retry. The facilitator or network rejects the payment because the wallet cannot fund it. The backend emits an error event, and the browser records a failed incident instead of showing a fake success.

This is important because a real system must show failures honestly. A failed payment is not a UI crash; it is an outcome that the agent can report and potentially recover from.

What the current page sections mean

Agent workspace

This is the main live demo. Run the autonomous task here. It contains the incident, TestNet toggle, underfunded-wallet toggle, event trace, settlement inspector, and returned patch.

Execution trace

This is the timeline of facts received from the backend. Every row has a real timestamp and an actor, such as Agent A, the gateway, the facilitator, or Agent B.

Settlement inspector

This is the payment receipt area. It shows the protocol, facilitator, network, asset, amount, payer, provider, and transaction link.

Incident ledger

This is local browser history for the incident flow. It records successful and failed runs, their timestamp, cost, reason, and time from incident detection to returned patch. The values come from the streamed events; it is not a list of fake sample summaries.

Capability catalog

This is the agent’s decision surface, not a human storefront. It shows what the agent can discover. After a live run, the capability selected in the actual catalog event is marked as selected for that run.

How to run the demo

From the repository root:

```
pnpm install
Set-Location .\server
```

```
pnpm start
```

Open:

```
http://localhost:4021
```

For judging, keep the TestNet checkbox enabled. Click Run Autonomous Task and watch the trace. When settlement completes, click the transaction ID and show the matching Lora page.

For the failure demonstration, enable Underfunded wallet and run again. The same flow should end in a clearly marked failure event and a failed ledger entry.

Useful verification commands

From the server directory:

```
pnpm typecheck
pnpm test
pnpm test:pay
pnpm test:pay:resolve-incident
```

What the payment smoke test proves:

```
unpaid request -> HTTP 402 -> signed payment -> facilitator settlement -> HTTP 200
```

How to explain the project in 60 seconds

“Threshold is a payment and discovery layer for software agents. When an agent cannot fix an incident, it queries a catalog of capabilities instead of relying on a human to find the right service. In our demo it selects an incident-resolution provider for a real tenant-isolation cache bug. The protected endpoint first returns HTTP 402. The server-side x402 client signs a 1000 micro-USDC payment on Algorand TestNet, GoPlausible verifies and settles it, and the request is retried with payment proof. The provider returns a structured diagnosis and unified diff. The browser shows the real event stream, settlement data, transaction ID, and Lora link. An underfunded-wallet toggle demonstrates the real failure path.”

Questions judges may ask

Why not just call Gemini directly?

Because the point is not only the AI answer. Threshold demonstrates a marketplace where an agent can discover separate capabilities and pay for them programmatically. Gemini is one provider behind the incident capability; x402 is the payment and access protocol.

Why does the payment happen on the server?

This is a controlled prototype. The server holds the demo payer so the browser never receives private keys. A production version should use a user-authorized or agent-authorized wallet with spending limits.

Is this real money?

The judging path is Algorand TestNet, so it uses test USDC rather than real money. The blockchain transaction and payment mechanics are real, but TestNet avoids financial risk during evaluation. MainNet configuration exists separately.

How do you know payment really happened?

The facilitator returns a settlement response containing success: true and a transaction ID. That ID is shown in the UI and opens on Algorand Lora, where the asset transfer can be independently inspected.

What stops one tenant from seeing another tenant's profile?

The incident demonstrates the bug and the fix: the cache key must include both tenant ID and user ID. The provider returns a patch changing userId to ${tenantId}:${userId}.

What happens if payment fails?

The same flow emits an error event and records a failed incident. The underfunded-wallet toggle intentionally exercises that path.

Is capability selection hardcoded?

The flow fetches the catalog, searches capabilities, selects the entry matching bug-resolution, and reports alternatives. The catalog contains incident resolution, code review, and summarization.

What is the strongest limitation?

This is a prototype. It uses a server-controlled payer, local browser history, and a single controlled incident fixture. Production use would need wallet authorization, durable records, provider authentication, spending limits, retries, and stronger patch verification.

Honest limitations

Do not claim that the browser applies the returned patch to a real repository. The current demo returns and displays a provider-generated patch and verification command. It proves the payment and structured-response loop. A future version could apply the patch in a sandbox and run the test automatically.

Do not call TestNet money real money. Say that the transaction is real on TestNet and uses test USDC.

Do not say the provider is a human debugging team. Agent B is the provider capability represented by the protected API and its AI-backed response.

File map

- server/index.ts: starts Hono, protects routes, configures x402 and GoPlausible.
- server/src/config/payment.ts: chooses network and USDC asset.
- server/src/catalog/apis.ts: returns discoverable capabilities.
- server/src/services/payment/incident-flow.ts: one real incident-payment flow used by terminal and browser.
- server/src/routes/run-payment-flow.ts: exposes that flow as SSE.
- server/src/routes/resolve-incident.ts: paid incident provider endpoint.
- server/src/services/ai/gemini.ts: asks Gemini for structured debugging output.
- server/public/app.js: reads SSE and updates the page.
- server/public/index.html: page structure and controls.
- server/public/style.css: visual design.
- .env: local secrets and network configuration; never commit it.

Final mental model

Think of Threshold as a vending machine for agent capabilities:

- The catalog is the menu.
- The 402 response is the price label.
- x402 is the payment mechanism.
- Algorand is the payment rail.
- GoPlausible is the payment verifier and settlement service.
- Agent B is the capability provider.
- The structured patch is the product delivered.
- The event trace is the receipt showing what actually happened.

That is the whole project: an agent discovers a useful service, pays for exactly one use, receives a machine-readable result, and can continue working without waiting for a human.